

FAST MARCHING

CALCUL DE CHEMINS MINIMAUX ET TRAITEMENT D'IMAGES

Cheng Léo, Garnaud Léo,
Fiolle Alexandre, Barrat Hugo

INTRODUCTIONS ET OBJECTIFS

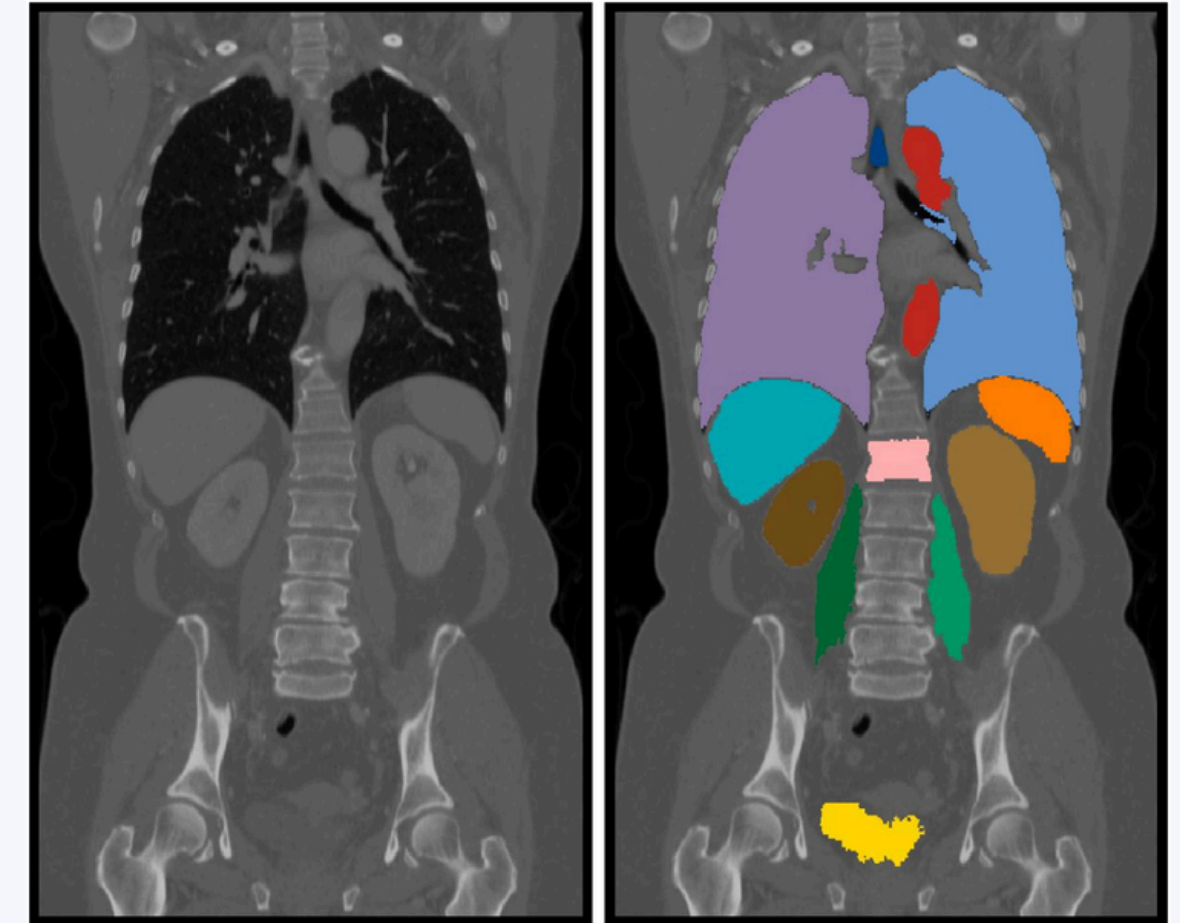
Contexte: Le Fast Marching est une méthode de calcul de chemins minimaux sur une carte de potentiel.

OBJECTIFS INITIAUX DU PROJET :

- Trouver le plus court chemin entre deux points sur une image (ex: photo satellite).
- Appliquer la méthode à la détection de contours.
- Réaliser une segmentation d'image multi-zones.

OBJECTIFS AJOUTES

- Développer une interface graphique pour faciliter l'usage des méthodes
- Minimiser les temps de calcul avec des meilleurs choix de structures et bibliothèques



Segmentation d'image médicale

CHOIX TECHNIQUES



LANGAGE DE PROGRAMMATION:

- Choix de Python plutôt que C++ en raison des nombreuses bibliothèques et du cours qui se basait sur ce langage, notamment pour les questions d'interface et de parallélisation

BIBLIOTHEQUES :

- PyQt6: Widgets, Gui, Core → interface graphique
- multiprocessing: Pool → parallélisation
- Pillow → gestion d'images

CLASSES DE MANIPULATION: Point, GreyImage, NumpyDict, PriorityQueue_heap

MODELISATION THEORIQUE

- Approche Graphe : l'image est considérée comme un graphe où les pixels sont des nœuds et les voisins sont liés par des arêtes.

- La Métrique $W(x,y)$:

Le coût de déplacement dépend du niveau de gris de l'image:

$$W(x, y) = \varepsilon + |f(x_0, y_0) - f(x, y)|$$

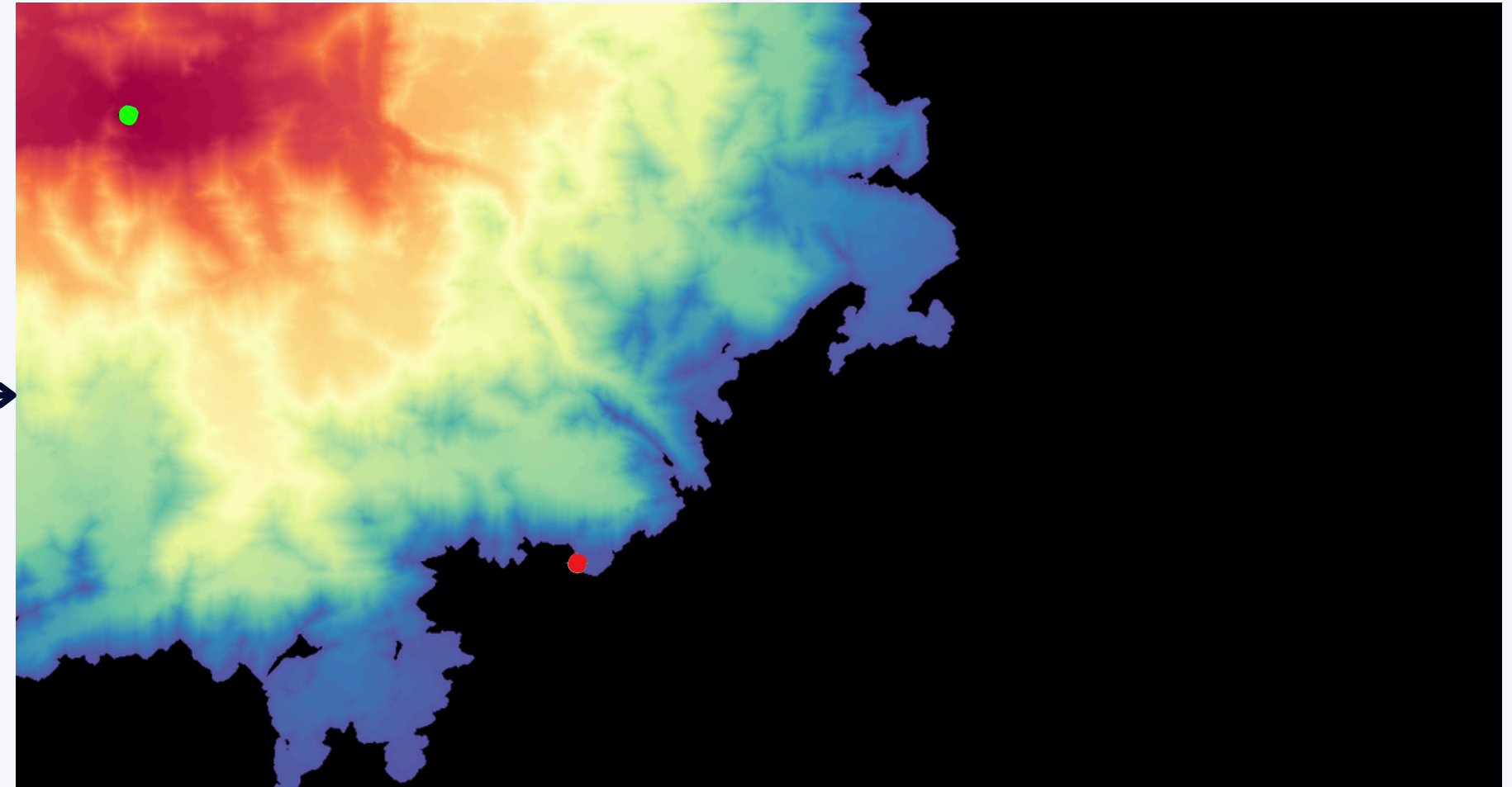
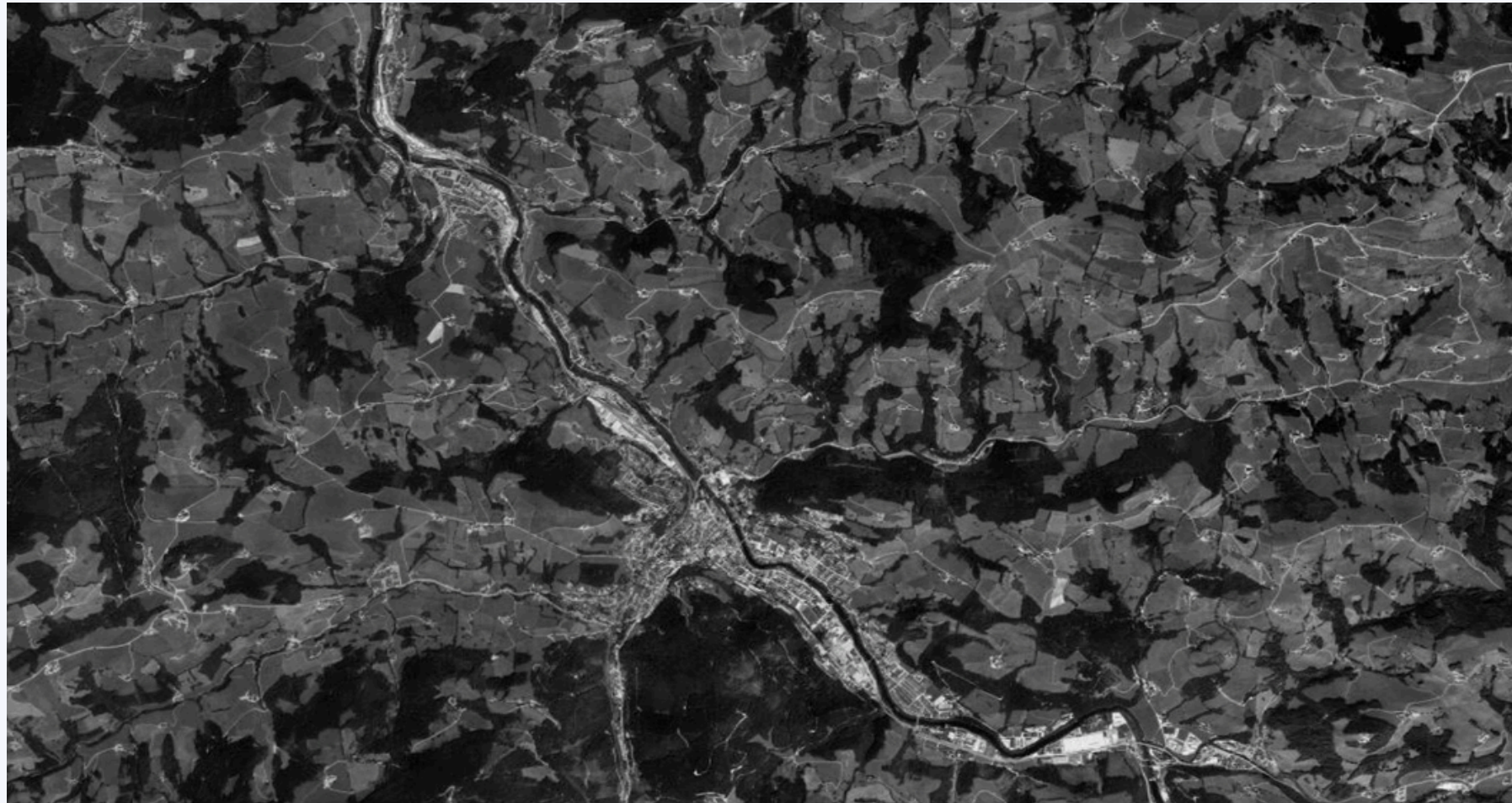
- ε est une constante de lissage et $f(x,y)$ représente le niveau de gris.
- Le coût du chemin :

$$\|L(r)\| = \int_0^1 (W(r(t)) \|r'(t)\|) dt$$

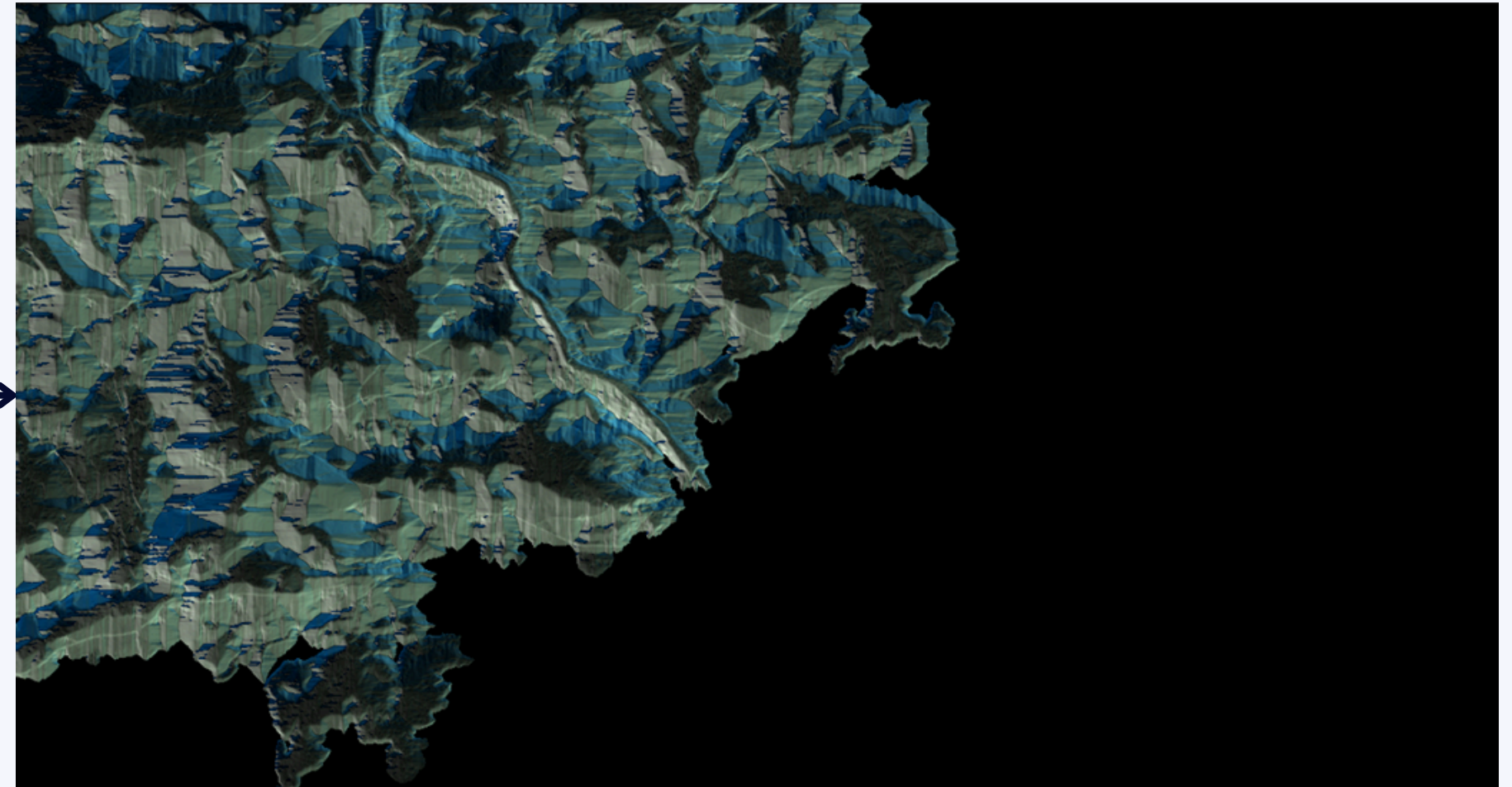
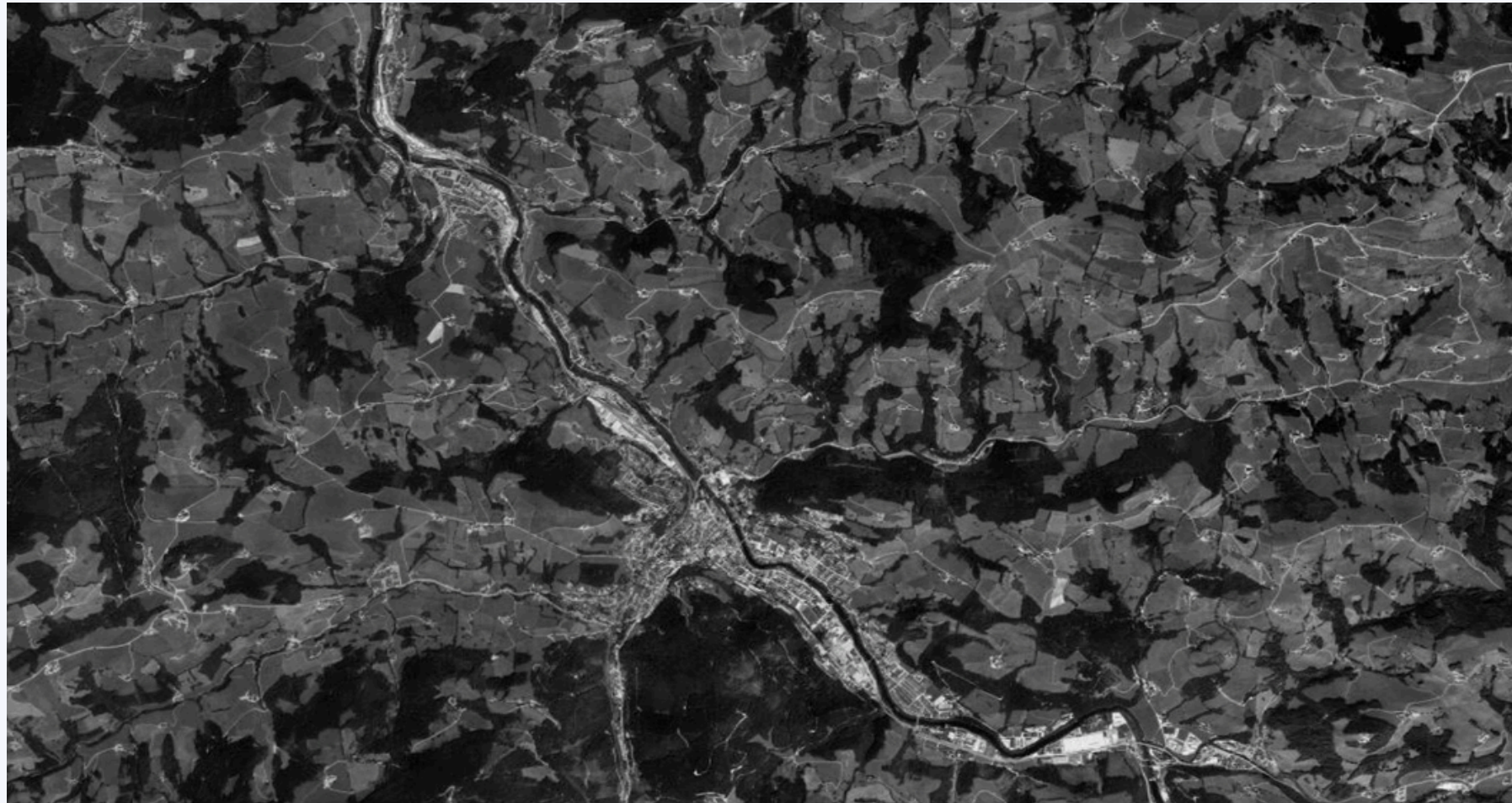
CARTE DE DISTANCE

- **Principe:** On propage un front depuis le point de départ en calculant le coût cumulé pour atteindre chaque pixel.
- **Propagation:**
Mise à jour itérative des voisins (et des voisins des voisins).
L'arrêt se fait quand le front touche le point d'arrivée.

CARTE DE DISTANCE



CARTE DES GRADIENTS



Pour (x, y) ne se situant pas sur le bord :

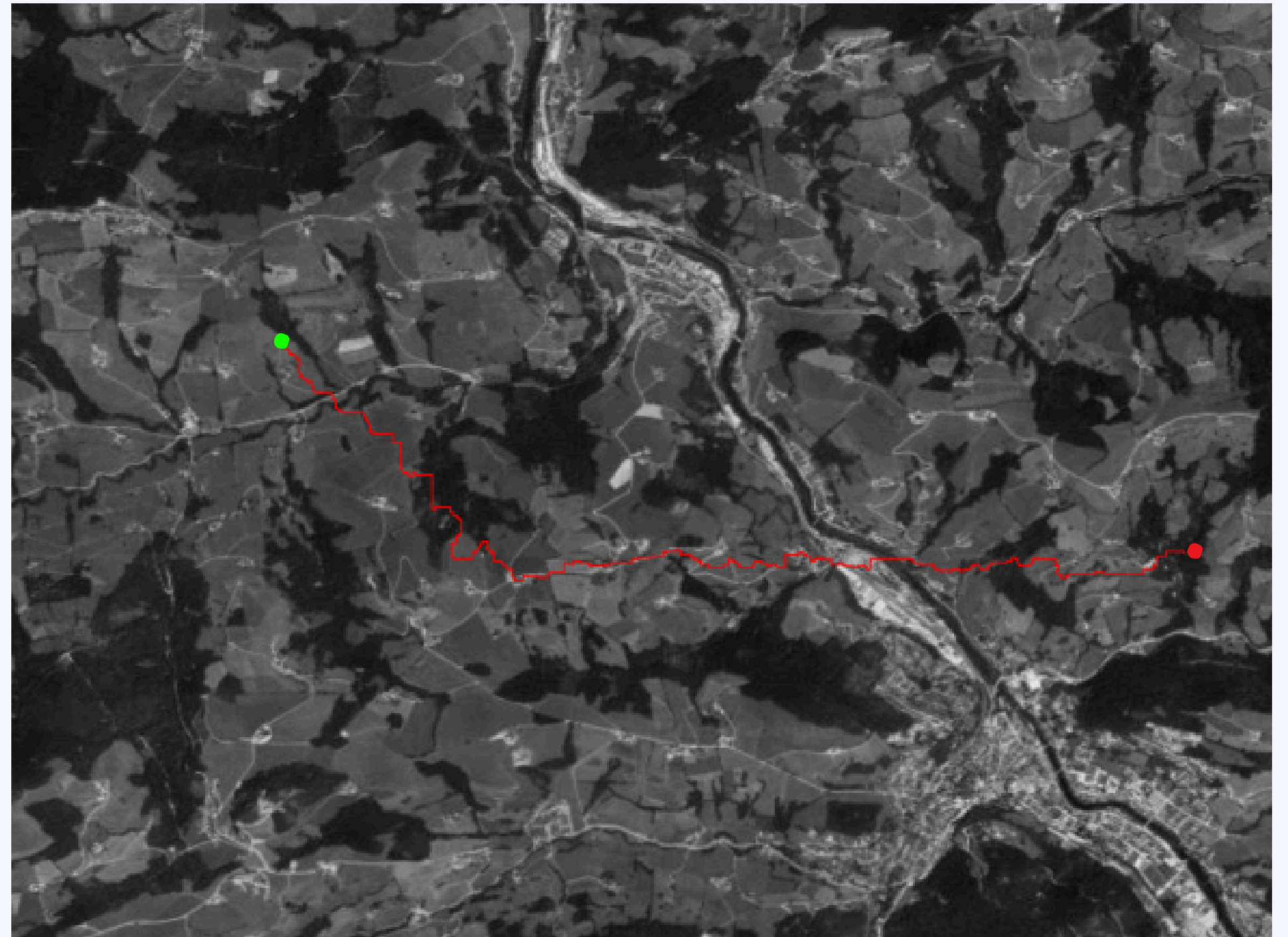
$$\text{grad}_x((x, y)) \approx \frac{\text{dist}((x+1, y)) - \text{dist}((x-1, y))}{2}$$
$$\text{grad}_y((x, y)) \approx \frac{\text{dist}((x, y+1)) - \text{dist}((x, y-1))}{2}$$

DESCENTE DE GRADIENT

Gradient "classique"

Algorithme de descente de gradient :

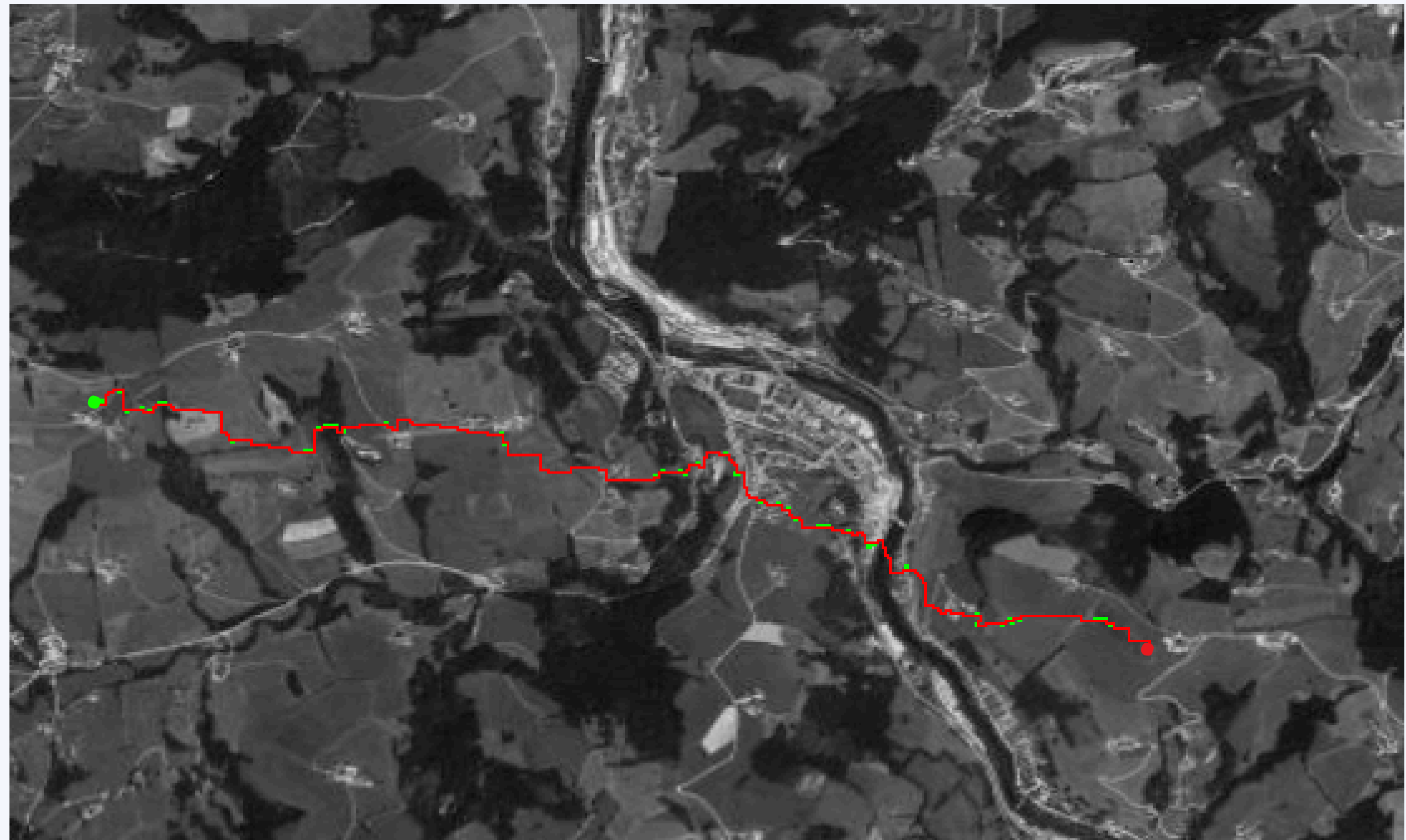
1. Démarrer du point d'arrivée,
2. Tant que le point de départ n'est pas atteint:
 - a. Calculer la liste des voisins atteignables,
 - b. Regarder l'opposé de la direction du gradient et prendre le point dans cette direction,
 - c. Ajouter ce point au chemin,
3. Retourner le chemin obtenu.



Chemin reconstruit en réalisant une descente de gradient

REMARQUES

- Lors de nos premières descentes de gradient, présence de boucles sur le chemin => ajout d'un algorithme de débouclage
- Ajout d'une fonction pour améliorer localement le gradient initial
- Temps de calcul pour le gradient long car passage par les distances



En rouge le chemin final

En vert la différence entre le chemin initial (avant l'amélioration) et le chemin final

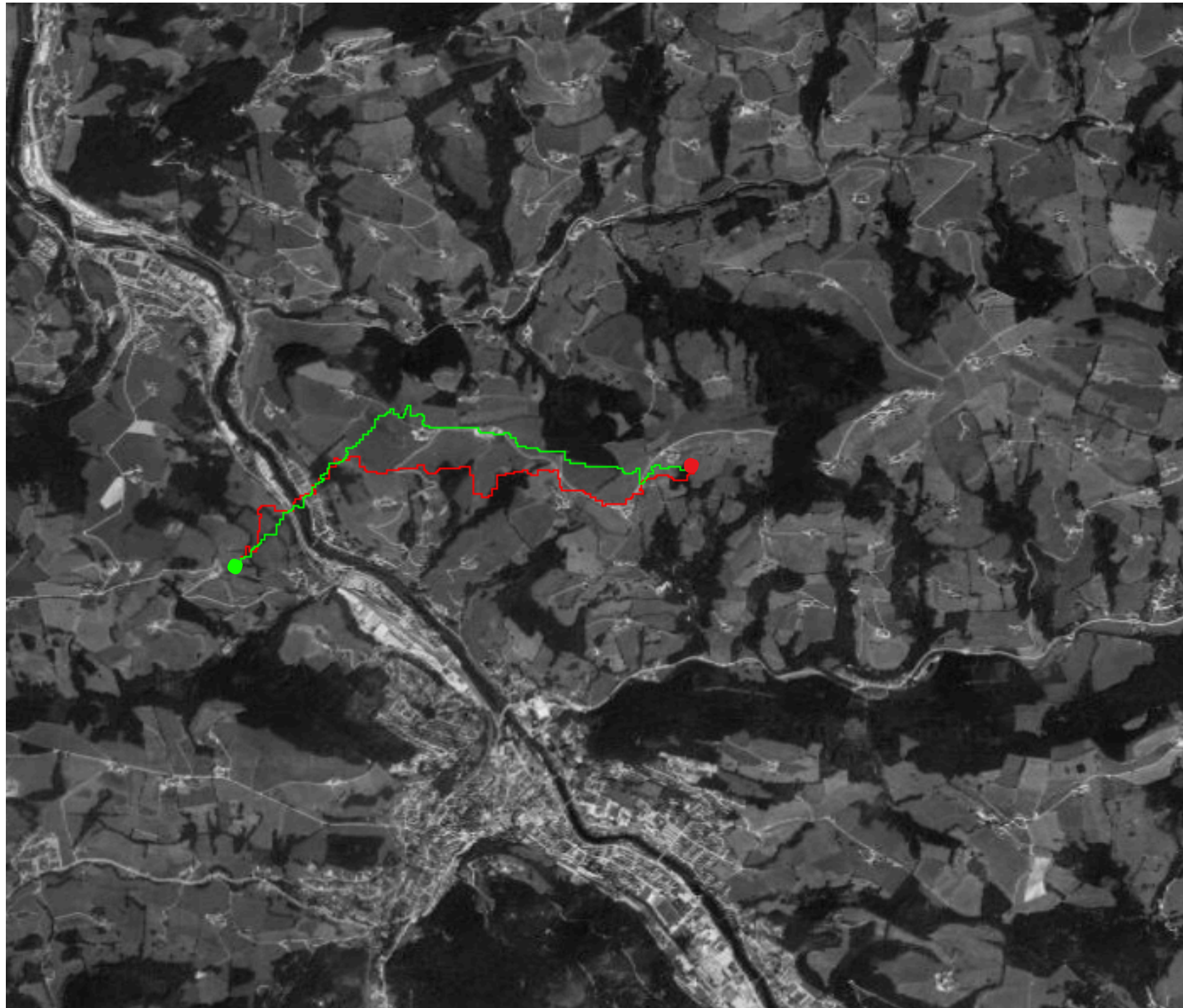
DESCENTE DE GRADIENT

Gradient de "Sobel"



Gradients de Sobel : convolution sur l'image par un filtre →
meilleure prise en compte de tout le voisinage (smooth)

COMPARAISON



- En vert : le chemin calculé avec le gradient de Sobel
 - Temps d'exécution de l'ordre de 5 secondes
 - Longueur du chemin : 463
 - Coût du chemin : 3195
- En rouge : le chemin calculé avec le gradient classique
 - Temps d'exécution de l'ordre de 20 secondes
 - Longueur du chemin : 485
 - Coût du chemin : 4425

INTERFACE

Bibliothèque utilisée: PyQt

Fonctionnalités développées :

- Chargement d'image et choix du mode (Chemin, Contour, Segmentation)
- Interactivité : Pose de points (départ/arrivée) à la souris, Annuler/Recommencer
- Visualisation : Affichage de la solution et sauvegarde des résultats
- Barre de chargement

DÉTECTION DE CONTOUR

1. Ligne ou surface qui limite extérieurement un corps, qui permet de le définir, de le déterminer, de le préciser ;
tour, forme : Une statue aux contours harmonieux.

SYNONYMES :

bord - linéament

Intuitivement: contour = zone (souvent une ligne courbée) délimitant deux zones de l'espace aux niveaux de gris "assez différents". Un contour est généralement fermé.

Lien avec des plus courts chemins ?

Choisir une métrique qui confère un coût proche de 0 sur des zones frontières de niveau de gris assez différents, et important sur des zones avec des niveaux de gris homogènes.

Placer deux points sur le contour, le plus court chemin restera donc sur le contour.

DÉTECTION DE CONTOUR

Métrique utilisée :

$$W(x, y) = \frac{1}{\varepsilon + G\sigma * G(x, y)}$$

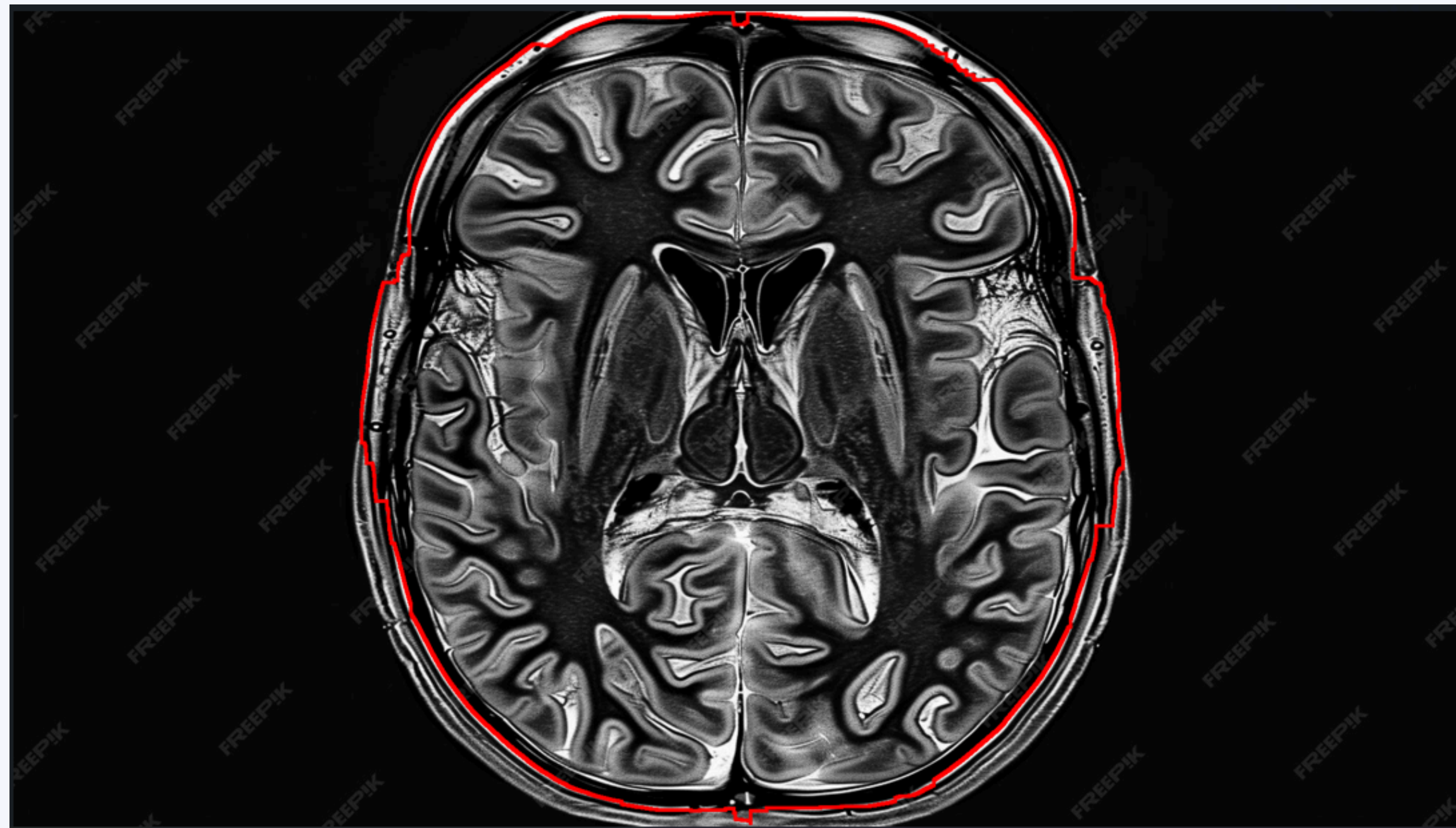
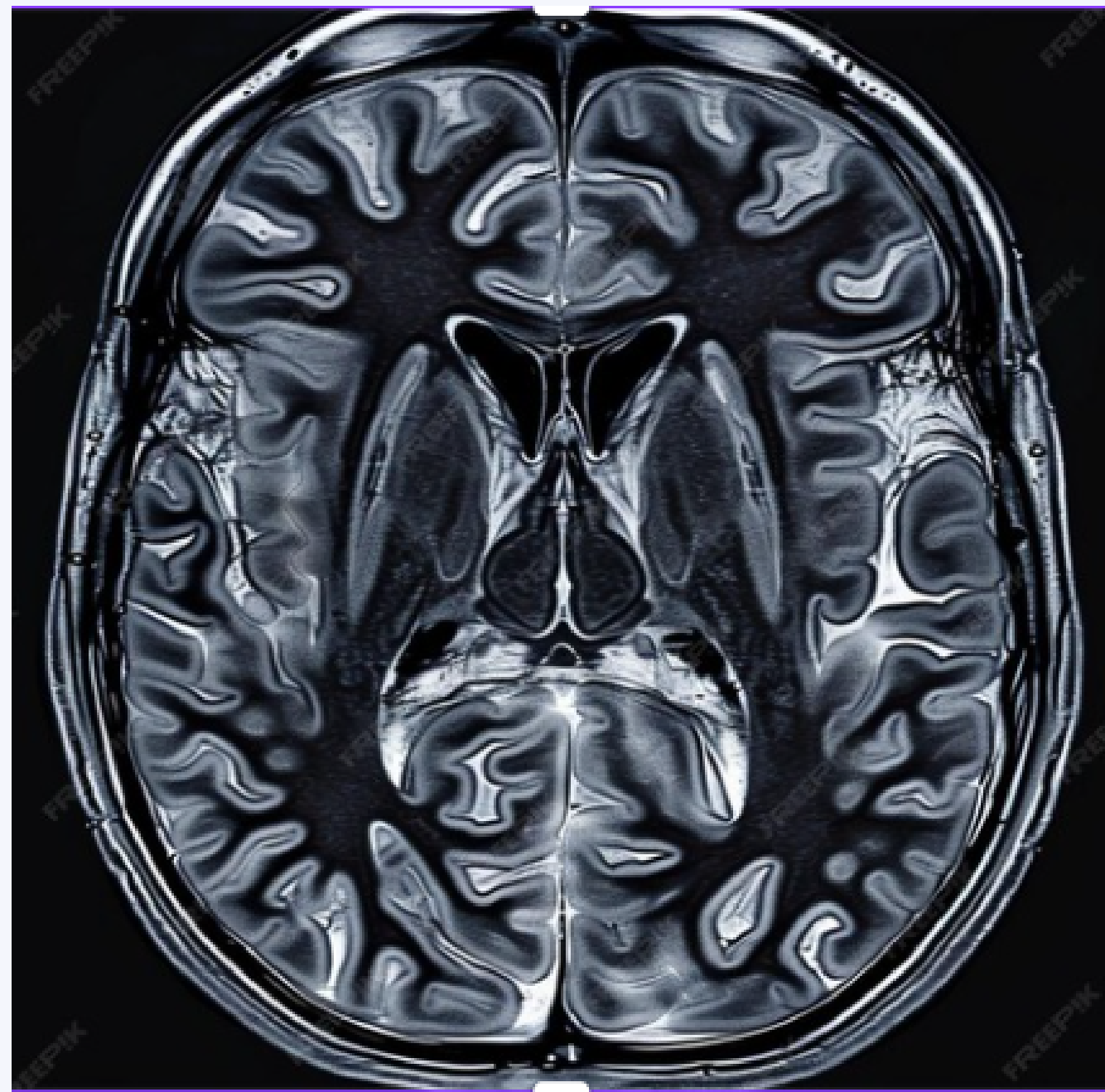
Pourquoi elle semble convenir :

(x, y) sur un contour $\Leftrightarrow$ Variation forte de niveaux de gris autour de (x, y)

$\Leftrightarrow$ Gradient $G(x, y)$ grand $\Leftrightarrow W(x, y)$ faible

Attention : on ne trace qu'une partie du contour. Il faut que l'algorithme place en arrière-plan de nouveaux points pour qu'il puisse faire le tour du contour.

DÉTECTION DE CONTOUR



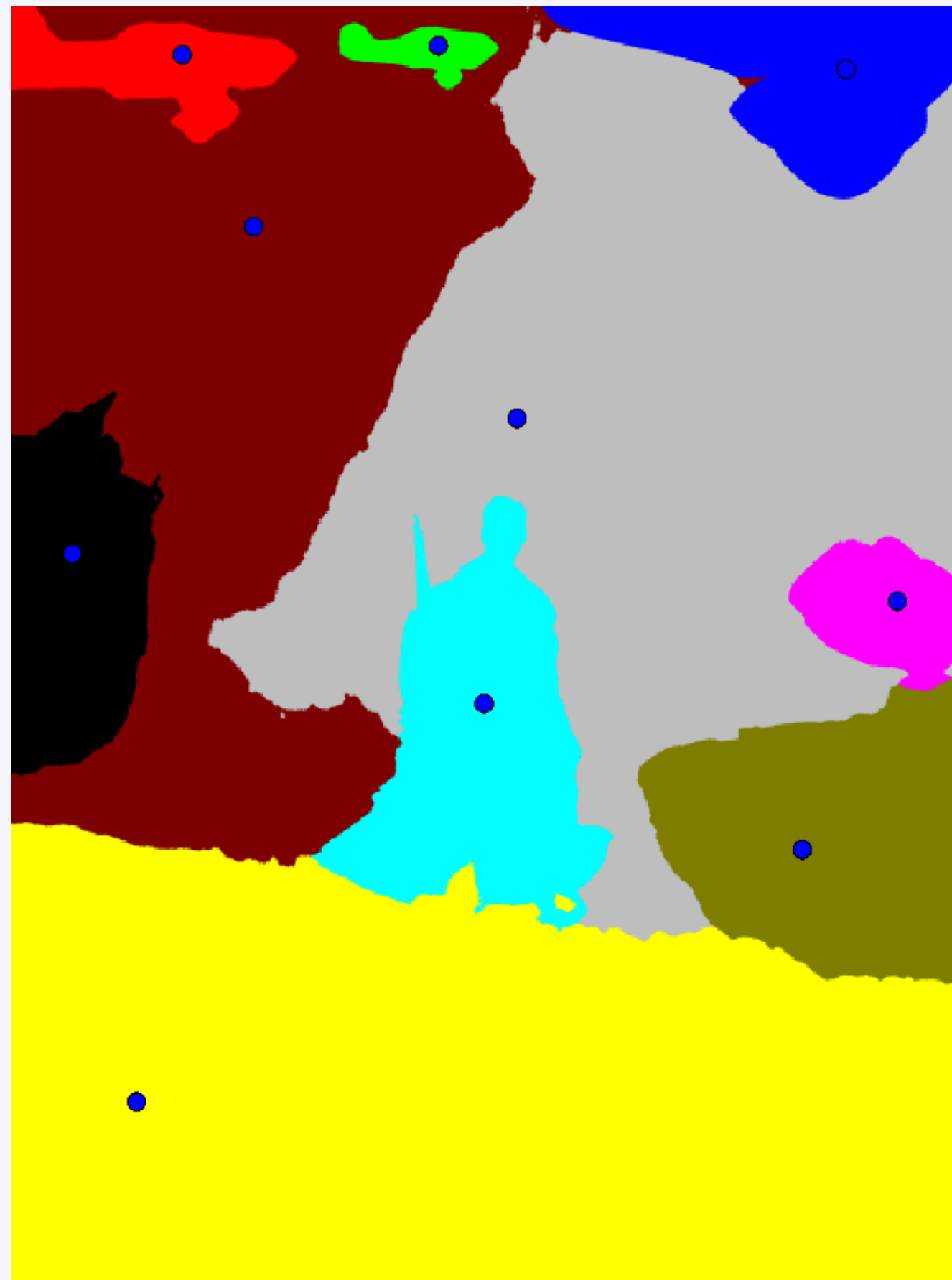
SEGMENTATION D'IMAGE

L'utilisateur sélectionne plusieurs points sur l'image qui serviront de base pour la segmentation.

On calcule ensuite les cartes des distances pour chaque point. Les calculs étant indépendants, on a décidé de paralléliser.

Ensuite on regarde pour chaque pixel de l'image de quel point la distance est la plus faible.

SEGMENTATION D'IMAGE



DIFFICULTÉS RENCONTRÉES

- Descente de gradient codée en dur : optimisation locale ne prenant en compte que les quatre directions cardinales, et pas l'ensemble du "voisinage" → remontée des gradients saccadée et lente
- Interface : se décompose en plusieurs classes interdépendantes (Vue, Menu, Chargement, Fenetre, Window → beaucoup de précautions nécessaires et de réflexion pour que tout communique correctement
- Interface : tester toutes les combinaisons de boutons les plus improbables pour déceler des comportements non souhaités et bloquer certaines fonctionnalités à certains moments
- Travailler en groupe : premier projet informatique de cette ampleur avec un groupe de quatre → difficultés de synchronisation, blocage si du retard est pris à un endroit, conflits VSCode...

UTILISATION DE L'IA

- Compléter du code "basique", exemple : boutons de l'interface pour afficher les cartes tous relativement similaires donc sur la fin complétés par le Co-pilot
- Trouver des erreurs précises dans le code (coquilles dans un long code, problèmes liées à des manques de maîtrise pointue de certaines bibliothèques)

OUVERTURE

- prolongement technique:
 - Interface plus intuitive
 - Emploi d'un langage compilé pour un calcul plus rapide
- approfondissement théorique: Tester d'autres métriques ou relier à l'équation eikonale pour d'autres applications
 - calcul du temps de propagation d'ondes sismiques
 - navigation robotique
 - voyageur de commerce continu